

Final Report

Course: Web Data and Semantics

Project: Knowledge Graph Construction, Alignment, Reasoning, KGE and RAG

Subject: NASA Artemis Program

Done by: Ali ABOUHACHIM ALAMI and Youssef BENADDI

GitHub Link: https://github.com/youss-ben/web_datamining_project

1. Data Acquisition and Information Extraction

1.1 Domain and seed URLs

The project focuses on the Artemis program, a strong candidate for a complete web data pipeline because it naturally combines organizations, missions, spacecraft, astronauts, places, and dates. It is therefore well suited to a workflow going from web crawling to question answering over an RDF graph.

The crawler defined in `src/crawl/crawler.py` starts from **20 seed URLs**, grouped into four source families:

- Planetary Society
- Space.com
- ESA
- NASA

The idea is to start from a small set of reliable pages, then discover nearby links that remain relevant to the Artemis domain.

1.2 Crawler design and ethics

The crawler implements a deliberately conservative collection strategy:

- `robots.txt` is checked for each domain using `urllib.robotparser`;
- a custom user-agent is declared: `ArtemisKB-Crawler/1.0 (Student Lab Project)`;
- a **1-second** delay is enforced between requests;
- link following is limited to **depth 1**;
- URLs are filtered using domain keywords;
- pages shorter than **200 words** are discarded;
- the crawl stops after **50 saved pages**.

These choices follow reasonable crawling practices and keep the process reproducible and proportionate for a course project.

The final corpus stored in `data/crawler_output.jsonl` contains:

Metric	Value
Saved pages	50
Minimum page length	204 words
Maximum page length	2886 words
Average page length	1130.18 words

1.3 Cleaning pipeline

The IE stage is implemented in `src/ie/ner_extraction.py` with spaCy `en_core_web_sm`.

The pipeline applies the following operations:

1. remove separators such as `|`;
2. remove long dash separators;
3. remove bracketed citations such as `[1]`;
4. normalize whitespace;
5. run sentence segmentation and NER;
6. extract subject-verb-object patterns from dependency parses;
7. keep only entities of type `ORG`, `PERSON`, `GPE`, `DATE`, and `LOC`.

This stage produces `data/extracted_knowledge.csv`, which contains **135 extracted triples**.

1.4 Examples of NER and relation extraction

Examples directly taken from `data/extracted_knowledge.csv`:

Head	Relation	Tail	Interpretation
NASA	launch	Artemis	NASA launches Artemis
NASA	launch on	November 16	extracted launch date
NASA	announce In	December 2020	announcement date
Reid Wiseman	announce In	August 2022	astronaut-related announcement

These examples show that the extraction stage recovers useful facts, but predicate normalization remains fragile, especially because of variations in casing and prepositions.

1.5 Three ambiguity cases

The PDF explicitly requires three ambiguity cases. In this project, the most important ones are the following:

1. 'Moon' detected as 'PERSON'

A manual correction already exists in `refine_entity()`: if the text is `moon` and the label is `PERSON`, it is remapped to `LOC`. This is a good example of a simple but effective domain-specific rule.

2. Relation fragmentation caused by casing and syntax

The CSV contains both `announce in` and `announce in`, as well as variants such as `tell on` and `announce on`. These differences create multiple RDF predicates for nearly equivalent actions and later degrade alignment, KGE, and RAG.

3. Boilerplate and article authors treated as domain entities

Entities such as `Mike Wall`, `Josh Dinner`, `Robert Z. Pearlman`, `Space.com`, or even `the day` appear in the extracted data. They come from editorial metadata or narrative text rather than from the core Artemis domain. This is one of the main noise sources in the pipeline.

2. KB Construction and Alignment

2.1 RDF modeling choices

Graph construction is handled in `src/kg/kb_construction.py`.

Main design choices:

- private namespace: `http://example.org/private_kb/`;
- entities are converted into URIs by replacing spaces with underscores and then URL-encoding the result;
- textual relations are converted into camelCase-like predicates;
- dates and long strings are kept as literals;
- other objects are converted to URIs.

The IE stage provides **135 factual triples**. After adding entity and predicate alignments, `kg_artifacts/ontology.ttl` contains:

Metric	Value
Triples	258

Metric	Value
Entities	335
Relations	92

2.2 Entity linking with confidence

Entity linking relies on the Wikidata search API (`wbsearchentities`). For each candidate entity:

- the first returned result is selected;
- confidence is set to `0.9` when a match exists;
- confidence is set to `0.0` otherwise.

The file `data/mapping_table.csv` reports:

Metric	Value
Candidate entities	98
Linked entities	73
Coverage	74.49%

Examples of successful links:

Private entity	Wikidata URI
Gateway	<code>http://www.wikidata.org/entity/Q53060</code>
Earth	<code>http://www.wikidata.org/entity/Q2</code>
The United States	<code>http://www.wikidata.org/entity/Q30</code>

Examples of unmatched entities:

Private entity	Status
Artemis Base Camp	no match
the Space Launch System	no match
the Orion Program	no match
the Starship Human Landing System	no match

This strategy is simple and reproducible, but the confidence score is clearly too optimistic: 0.9 only means "first result found", not "reliable match".

2.3 Predicate alignment

Predicate alignment is also handled in `src/kg/kb_construction.py`, this time through a SPARQL query over Wikidata properties.

The file `data/predicate_alignment.csv` shows:

Metric	Value
Private predicates	92
Aligned predicates	71
Coverage	77.17%

Some useful alignments are plausible:

Private predicate	Wikidata property	Meaning
launch	wdt:P619	UTC date of spacecraft launch
retireAfter	wdt:P730	date of service retirement
beAfter	wdt:P138	named after

However, other mappings are clearly weak or off-topic:

- spend -> minimum spend bonus
- change over -> stock exchange
- orbit -> time of object orbit decay

So the numerical coverage is acceptable, but the true semantic precision is lower than the raw percentage suggests.

2.4 Expansion strategy

Expansion is implemented in `src/kg/kb_expansion.py`.

The strategy is a Wikidata snowball expansion:

- start from entities linked through `owl:sameAs`;
- query only direct `wdt:` properties;
- keep only IRI objects;
- exclude media/file links;

- limit each query to 50 outgoing triples;
- cap the full graph at **65000 triples**, **20000 entities**, and **120 relations**.

This method makes it possible to reach a graph size suitable for KGE while still keeping some control over graph growth.

2.5 Final KB statistics

The final graph stored in `kg_artifacts/expanded.nt` and summarized in `data/statistics_report.txt` contains:

Metric	Value	PDF target
Triples	65014	50k-200k
Entities	18190	large enough for KGE and RAG
Relations	120	within the requested 50-200 range

These are the real values present in the repository. They replace the earlier estimates from the initial draft.

3. SWRL Reasoning

3.1 SWRL rule on `family.owl`

Ontology generation is implemented in `src/reason/create_ontology.py`, and reasoning is handled in `src/reason/swrl_reasoning.py`.

The rule applied on the family ontology is:

```
Person(?p), hasAge(?p, ?a), greaterThan(?a, 65) -> oldPerson(?p)
```

Observed output:

- `Michael (69) -> oldPerson`
- `Peter (70) -> oldPerson`

The inferred ontology is saved in `data/family_inferred.owl`.

3.2 SWRL rules on the Artemis ontology

For the project itself, the script defines a small domain ontology with missions, agencies, launch vehicles, and crew members.

Two rules are applied:

```
SpaceMission(?m), operatedBy(?m, ?a), usesVehicle(?m, ?v) ->
manages(?a, ?v)
SpaceMission(?m), operatedBy(?m, ?a), hasCrew(?m, ?c) ->
trainsCrewFor(?a, ?c)
```

Observed outputs:

- NASA manages SLS
- NASA manages Orion
- NASA manages Starship_HLS
- NASA trains Reid_Wiseman
- NASA trains Victor_Glover

The resulting ontology is saved in `kg_artifacts/artemis_domain.owl`.

The code first tries Pellet/HermiT, then falls back to a manual Python application of the rules when Java is unavailable. This fallback is practical and improves reproducibility.

4. Knowledge Graph Embeddings

4.1 Data cleaning and splits

The KGE work is contained in `notebooks/TD5_Knowledge_Reasoning.ipynb`.

The notebook explicitly states that the original graph is **augmented/duplicated** to satisfy the graph size requirement of the assignment. This must be mentioned because it directly affects result interpretation.

The split files generated in `data/` contain:

File	Number of triples
train.txt	48353
valid.txt	6044
test.txt	6044

This gives a total of **60441 triples** for the KGE dataset.

Additional subsets are also present:

File	Size
train_small.txt	10571
train_10k.txt	12085
train_20k.txt	20000

4.2 Trained models

The notebook trains four models, which is more than the minimum of two required in the PDF:

- TransE
- ComplEx
- DistMult
- RotatE

4.3 Metrics: MRR and Hits@k

The metrics extracted from the notebook outputs are:

Model	MRR	Hits@1	Hits@3	Hits@10
TransE	0.0445	0.0003	0.0566	0.1257
ComplEx	0.1694	0.0530	0.2501	0.3692
DistMult	0.1535	0.0446	0.1959	0.3716
RotatE	0.2613	0.1634	0.3082	0.4595

The best model in this repository is therefore **RotatE**.

Interpretation:

- TransE performs clearly worse, which is expected on a heterogeneous and noisy graph;
- ComplEx and DistMult both improve the scores significantly;
- RotatE obtains the best overall performance.

4.4 Size sensitivity

The PDF asks for a 20k / 50k / full size analysis. In the notebook, the comparison that is actually available is:

Training size	MRR
20000	0.1111
48353	0.2511

The project does not contain a distinct run exactly at 50k. In practice, the real maximum available size, **48353**, acts here as the "50k/full" condition.

The general trend is still clear: more data improves the embeddings, even though graph noise limits the final semantic quality.

4.5 t-SNE and nearest neighbors

The notebook already contains an embedded t-SNE figure, which was extracted as a report artifact:

A script version also exists in `src/kge/tsne_analysis.py`, using a DistMult-based visualization with KMeans.

The nearest neighbors shown in the notebook are particularly informative because they reveal graph noise directly:

- neighbors of Artemis: Lyan, Julien, Albert Eduard, division in classification of productive activities, Jules
- neighbors of a noisy NASA-related label: NASA, Category:The Bronx, Category:Competition, Category:Society

These neighborhoods are not semantically satisfying for the target domain. They show that the embedding space absorbed noise coming from crawling, extraction, and alignment.

The notebook also includes a structural logic test:

```
announceIn(?a, ?b) AND announceOn(?b, ?c) -> astronaut(?a, ?c)
L2 Distance ||(r1 o r2) - r3|| : 22.7263
```

This suggests that some vector regularities are learned, but the result should be interpreted cautiously given the underlying graph quality.

5. RAG over RDF/SPARQL

5.1 Schema summary

The RAG pipeline is implemented in `src/rag/rag_pipeline.py`.

It loads `kg_artifacts/expanded.nt` and extracts:

- RDF prefixes;
- distinct predicates;
- classes;
- sample triples;
- a dictionary of known entities.

This schema summary is injected into the prompt so that the model only uses known predicates and URIs.

5.2 NL -> SPARQL prompt

The pipeline uses a local Ollama model (`llama3.1:8b`) and a tightly constrained prompt:

- only use URIs visible in the schema;
- do not invent external URIs;
- return a `SELECT` query;
- ensure that at least one answer variable remains unbound in the `WHERE` clause;
- return only a `sparql` block.

Five few-shot examples are provided, for example:

```
SELECT ?mission WHERE {
  <http://example.org/private_kb/NASA>
  <http://example.org/private_kb/launch> ?mission .
```

```
}
```

5.3 Self-repair mechanism

The pipeline includes a useful self-repair loop:

- detect queries without a real answer variable;
- catch syntax and execution errors;
- treat zero-result queries as partial failures;
- ask the model for a corrected query;
- retry up to two times.

This is one of the strongest points of the project because it makes the NL -> SPARQL step much more robust than a simple one-shot generation.

5.4 Evaluation on at least five questions

The file `data/rag_evaluation.json` contains an evaluation on **5 questions**, matching the PDF requirement.

Question	Baseline LLM	RAG result	Attem pts	Comment
What did NASA launch?	says context is missing	Artemis	1	correct and grounded
What does the SLS rocket do for Artemis?	generic explanation	Artemis via doFor	2	repaired query
What is NASA planning?	broad answer about Moon/Mars	Artemis_4	1	grounded
When was Artemis launched?	fluent answer about Artemis I	16 November 2022, Wednesday 16 November 2022	1	correct but duplicated date
Who is an astronaut for NASA?	generic definition of astronaut	Josh_Dinner, Robert_Z._Pearlman_Published -_Artemis	1	execution succeeds but the KB is polluted

Summary:

- **5/5** evaluations execute successfully in the end;
- **1/5** requires self-repair;
- compared with the baseline, the RAG system is better grounded in the graph;
- however, it also inherits the noise of the KB and may therefore return answers that are graph-grounded but semantically wrong.

6. Critical Reflection

6.1 Impact of KB quality

This project shows very clearly that noise from the early stages propagates throughout the pipeline:

- imperfect extraction creates noisy entities and relations;
- poorly normalized predicates degrade RDF modeling and alignment;
- weak entity linking causes drift during expansion;
- that drift then affects KGE;
- finally, RAG becomes well grounded in the graph, but not necessarily correct.

So the pipeline is technically complete, but upstream graph quality remains decisive.

6.2 Main noise sources

The most visible issues are:

1. Pollution from authors and editorial metadata

Names of journalists or websites are kept as entities.

2. Relation normalization problems

Variants such as `announce In` and `announce in` split the graph unnecessarily.

3. Over-aggressive alignment

The first Wikidata hit is not always the correct one.

4. Expansion drift

One poor anchor can move the expansion away from the Artemis core.

5. Canonicalization problems on the RAG side

For example, in `KNOWN_ENTITIES`, `moon` is mapped to `Harvest_Moon`, which clearly indicates that more cleanup is needed.

6.3 Symbolic reasoning versus embedding-based reasoning

Aspect	SWRL	KGE
Nature	symbolic and explicit	statistical and latent
Explainability	high	lower
Robustness to noise	better when the ontology is clean	sensitive to noisy graph structure
Generalization	limited to written rules	can capture softer regularities
Best use here	fact validation	link prediction and representation learning

In this repository, SWRL provides small but reliable inferences, while KGE gives broader results that are much more sensitive to noise. The two approaches are therefore complementary.

Conclusion

The repository satisfies the main technical goals of the assignment:

- web crawling with reasonable safeguards;
- information extraction;
- RDF graph construction;
- entity and predicate alignment;
- expansion beyond 50k triples;
- SWRL reasoning;
- KGE evaluation across several models;
- RAG with NL -> SPARQL generation and self-repair.

The main strengths are the end-to-end project structure, the presence of many reproducible artifacts, the final graph size, and the design of the RAG module with a correction loop.

The main limitation is graph quality: the graph is large enough, but not yet clean enough. This explains both the poor semantic neighbors observed in KGE and the fact that RAG can return answers that are grounded in the graph but still wrong in meaning. In other words, the project successfully demonstrates the full web data and semantics pipeline, and it also highlights the central lesson of the course: **graph quality matters as much as graph size**.